Password Hashing Algorithms

What exactly are the differences between PASSWORD_BCRYPT, PASSWORD_ARGON2I etc? We've briefly touched on what these algorithms exist to do: provide a one-way method of generating unique, fixed-length identifiers for strings and generally make life difficult for bad actors to bulk-replicate these identifiers. Blowfish/bcrypt, argon2i, and argon2id are all hashing algorithms designed to maximize cost: they deliberately consume memory and CPU resources. In particular, the bcrypt algorithm takes a parameter to control the number of iterations (or cycles) to perform: the idea is that as computing power increases in capability (and reduces in cost), the hashing algorithm can become incrementally more expensive to match. To illustrate, here are a few examples (using both password_hash and another core function, crypt()), and a little extra code to demonstrate how long the calls take.

```
$password = 'weaksauce';

$start = microtime(true);
$hash1 = password_hash($password, PASSWORD_DEFAULT);
$end = microtime(true) - $start;
print "{$hash1} - took {$end} s" . PHP_EOL;

Output:
$2y$10$2alN7H818NP.8BZAoerYxebJIOTsNz4MItmDx1notHgXQwNGxsSCC - took
0.052612066268921 s
```

Here's the same code using crypt(), which is more versatile than password_hash() – indeed the PHP internal implementation of password_hash() is a simple wrapper around crypt(). The $2y$10$ portion of the output tells us that the bcrypt algorithm was used with a cost parameter of 10. Let's create a simple wrapper function that we can use to control the cost and measure its effect. We can run the password string and the salt (extracted from the output of password_hash()) to prove that crypt() is doing the same internal work:

```
function doHash(string $password, string $salt, int $cost = 10): void
{
    $start = microtime(true);
    $hash = crypt($password, "$2y\${$cost}\${$salt}");
    $end = microtime(true) - $start;
    print "{$hash} - took {$end} s" . PHP_EOL;
}
```